Prompt:

I want you to create LTL constraints from sentences. the set of LTL constraints that you are allowed to use are the following five:

- A -> F(B), which means that when A occurs, then B must eventually occur afterwards

- NOT(B) U A, which means that B must not occur until A has occured

- (A -> F(B)) AND (NOT(B) U A), which combines both of the above, so A must occur before B every time

- START -> A, which means that the first symbol must be A

- END -> A, which means that the last symbol must be A

you may switch A and B at will, where A and B are the symbols of the LTL constraint

There can be more than one constraint in a sentence

Please wait for the sentences

additionally, please only use the following symbols to replace A and B in the sentences:

create claim

creation claim

approve claim

reject claim

payout

pay out claim

create case

arrange meeting

enter metadata

submit case

assign case id

agree date

examine patient

check x-ray risk

perform x-ray

perform surgery

apply cast

book ticket

payment ticket

check-in flight

receive invoice

decide invoice legit

receive all necessary approvals

confirm payment

receipt of an invoice

enter data

pay invoice

send request

give confirmation

register client data

provide room service

provide laundry service

leave

search report

bring in a defective computer

each of these activities

error is detected

register claim

Check recommendation

Mark as not ok

Mark as Ok

Receive request for new hardware

Analyze request

Deny request

Approve request

Receive offer

this is accepted

take care of invoice

receive customer request

need further review

submit office supply request

register requirement

reject request

ask to make change

select vendor

approval, rejection, change

submit vacation request

review order

inform customer

wait for x days

inform about delay

deliver package

Receive order

reserve part

ship bicycle to the customer

Receive amount request

Request approval

File expense

Receive expense report

check take place

supervisor approves

supervisor rejects

respond

approval claim

pay claim

case closed manually

manage case

hold meeting

inform

get local id

propose dates

perform x-ray risk

perform reposition

prescribe rehabilitation

remove cast

provide personal information

booking phase completion

money transfer

modify provided data

cancellation

enter all relevant data

close case

give approval

prepare proposal

proposal

bill

charge bill

put in a file together with the claim documentation

analyze the defect

test proper system functionality

execute repair activity

examine claim

send it back

Proceed with claim

inform employee via email

edit request

fill in form

decide if approved

receive hardware by IT department

Send back request to IT

receive request

return to petitioner/employee

go to purchase department

generate purchase order

send notification

return application

generate notification

order missing products

check again

process delivery

update system

assemble bicycle

Assess credit risk

Accept credit

Perform check

stop process

deposit of reimbursement

receives a notification

send email

please create LTL constraints for these 10 sentences:

A claim should be created, before it can be approved.

A claim must be created, before it is approved.

When a claim is created, it may be approved.

If a claim is created, it can be approved.

They have to create the claim, before they can approve it.

Only after a claim is created, it is possible to approve the claim.

If a claim is approved, then it must have been created first.

To approve a claim, it should be created first.

A claim can be approved, after it has been created.

Approving the claim can happen at anytime, unless it still needs to be created.

Response:
Here are your sentences translated into LTL constraints:

"A claim should be created, before it can be approved."

"create claim" -> F("approve claim")

"A claim must be created, before it is approved."

"create claim" -> F("approve claim")

"When a claim is created, it may be approved."

"create claim" -> F("approve claim")

"If a claim is created, it can be approved."

"create claim" -> F("approve claim")

"They have to create the claim, before they can approve it."

"create claim" -> F("approve claim")

"Only after a claim is created, it is possible to approve the claim."

"create claim" -> F("approve claim")

"If a claim is approved, then it must have been created first."

"approve claim" -> F("create claim") AND NOT("create claim") U "approve claim"

"To approve a claim, it should be created first."

"create claim" -> F("approve claim")

"A claim can be approved, after it has been created."

"create claim" -> F("approve claim")

"Approving the claim can happen at anytime, unless it still needs to be created."

NOT("create claim") U "approve claim"

Please note that these LTL constraints are simplified based on the context given and the limited set of constraints you provided. The actual implementation may vary based on the specific requirements of your system.

Prompt:
now these 10:

Creation of the claim is required, before it can be approved.

The creation of the claim is a condition for the approval of the claim.

Only after the creation of a claim, the approval of the claim is possible.

When approval of the claim occurs, creation of the claim must have preceded.

If approval of the claim occurs, then creation of the claim must have been carried out first.

The approval of the claim requires that the creation of the claim occurs before it.

The approval of the claim should be preceded by the creation of the claim.

A claim must be created, before it can be approved or rejected.

A claim should be approved before it can be paid out.

Once a claim has been approved, it should eventually be paid out.

Response:
Here are your sentences translated into LTL constraints:

"Creation of the claim is required, before it can be approved."

"creation claim" -> F("approve claim")

"The creation of the claim is a condition for the approval of the claim."

"creation claim" -> F("approve claim")

"Only after the creation of a claim, the approval of the claim is possible."

"creation claim" -> F("approve claim")

"When approval of the claim occurs, creation of the claim must have preceded."

"approve claim" -> F("creation claim") AND NOT("creation claim") U "approve claim"

"If approval of the claim occurs, then creation of the claim must have been carried out first."

"approve claim" -> F("creation claim") AND NOT("creation claim") U "approve claim"

"The approval of the claim requires that the creation of the claim occurs before it."

"creation claim" -> F("approve claim")

"The approval of the claim should be preceded by the creation of the claim."

"creation claim" -> F("approve claim")

"A claim must be created, before it can be approved or rejected."

"create claim" -> F("approve claim") AND F("reject claim")

"A claim should be approved before it can be paid out."

"approve claim" -> F("pay out claim")

"Once a claim has been approved, it should eventually be paid out."

"approve claim" -> F("pay out claim")

Again, please note that these LTL constraints are simplified based on the context given and the limited set of constraints you provided. The actual implementation may vary based on the specific requirements of your system.

Prompt:
now these 10:

Once a claim has been rejected, it should not be paid out.

Payout should end the process and therefore exclude everything.

A case can be closed manually at any time, unless we still need to pay out the claim.

The case can be managed at the union after it has been created.

After a case is created, management can and must arrange a meeting between the union case worker and the other case worker.

After a meeting is arranged it must be held.

The case worker should enter metadata on the case, inform about when he is available for participating in a meeting and then submit the case.

When a case is submitted it may get a local id at the union, but it should also subsequently be assigned a case id in LO.

When a case is submitted, LO should eventually propose dates.

Only after LO has assigned its case id it is possible to manage the case and for LO to propose dates.

Response:
Here are your sentences translated into LTL constraints:

"Once a claim has been rejected, it should not be paid out."

"reject claim" -> F(NOT("pay out claim"))

"Payout should end the process and therefore exclude everything."

"payout" -> F(END)

"A case can be closed manually at any time, unless we still need to pay out the claim."

NOT("pay out claim") U "case closed manually"

"The case can be managed at the union after it has been created."

"create case" -> F("manage case")

"After a case is created, management can and must arrange a meeting between the union case worker and the other case worker."

"create case" -> F("arrange meeting")

"After a meeting is arranged it must be held."

"arrange meeting" -> F("hold meeting")

"The case worker should enter metadata on the case, inform about when he is available for participating in a meeting and then submit the case."

"enter metadata" AND "inform" AND "submit case" -> F(START)

"When a case is submitted it may get a local id at the union, but it should also subsequently be assigned a case id in LO."

"submit case" -> F("get local id") AND F("assign case id")

"When a case is submitted, LO should eventually propose dates."

"submit case" -> F("propose dates")

"Only after LO has assigned its case id it is possible to manage the case and for LO to propose dates."

"assign case id" -> F("manage case") AND F("propose dates")

Please note that these LTL constraints are simplified based on the context given and the limited set of constraints you provided. The actual implementation may vary based on the specific requirements of your system.

Prompt:
now these 10:

Once a date has been agreed upon a meeting should eventually be held.

Every process instance starts by examining a patient.

If an x-ray is taken, then the x-ray risk must be checked before it

Performing a reposition, applying a cast and performing surgery require that x-rays are performed before.

If a surgery is performed, then prescription of rehabilitation occurs eventually after it.

After a cast is applied, eventually the cast is removed and vice versa, before every removal of a cast, a cast must be applied.

The process begins with the booking of the ticket

Personal information of passengers can be provided after the booking of the ticket

The payment of the ticket triggers the completion of the booking phase.

The payment of the ticket is eventually followed by the actual transfer of money

Response:
Here are your sentences translated into LTL constraints:

"Once a date has been agreed upon a meeting should eventually be held."

"agree date" -> F("hold meeting")

"Every process instance starts by examining a patient."

START -> "examine patient"

"If an x-ray is taken, then the x-ray risk must be checked before it."

"perform x-ray" -> F("check x-ray risk") AND NOT("check x-ray risk") U "perform x-ray"

"Performing a reposition, applying a cast and performing surgery require that x-rays are performed before."

"perform x-ray" -> F("perform reposition") AND F("apply cast") AND F("perform surgery")

"If a surgery is performed, then prescription of rehabilitation occurs eventually after it."

"perform surgery" -> F("prescribe rehabilitation")

"After a cast is applied, eventually the cast is removed and vice versa, before every removal of a cast, a cast must be applied."

"apply cast" -> F("remove cast") AND NOT("remove cast") U "apply cast"

"The process begins with the booking of the ticket"

START -> "book ticket"

"Personal information of passengers can be provided after the booking of the ticket"

"book ticket" -> F("provide personal information")

"The payment of the ticket triggers the completion of the booking phase."

"payment ticket" -> F("booking phase completion")

"The payment of the ticket is eventually followed by the actual transfer of money."

"payment ticket" -> F("money transfer")

Please note that these LTL constraints are simplified based on the context given and the limited set of constraints you provided. The actual implementation may vary based on the specific requirements of your system.

Prompt:
now these 10:

As long as the Check-in of the flight does not take place, the customers can still modify the provided data to change

After check-in of the flight, only the cancellation is admitted

The process starts when an invoice is received by the administration department.

If they decide that the invoice appears legit then they enter all relevant data into the system

When all necessary approvals have been received the invoice can be paid.

Once payment is confirmed, the invoice case should be closed.

The receipt of an invoice is required before the administration department can enter data or scan the invoice.

Enter data is required before any approval can be given.

Finally, we have to pay the invoice before we can confirm payment.

If a request is sent, then a proposal is expected to be prepared afterwards

Response:
Here are your sentences translated into LTL constraints:

"As long as the Check-in of the flight does not take place, the customers can still modify the provided data to change."

NOT("check-in flight") U "modify provided data"

"After check-in of the flight, only the cancellation is admitted."

"check-in flight" -> F("cancellation")

"The process starts when an invoice is received by the administration department."

START -> "receive invoice"

"If they decide that the invoice appears legit then they enter all relevant data into the system."

"decide invoice legit" -> F("enter all relevant data")

"When all necessary approvals have been received the invoice can be paid."

"receive all necessary approvals" -> F("pay invoice")

"Once payment is confirmed, the invoice case should be closed."

"confirm payment" -> F("close case")

"The receipt of an invoice is required before the administration department can enter data or scan the invoice."

"receipt of an invoice" -> F("enter data")

"Enter data is required before any approval can be given."

"enter data" -> F("give approval")

"Finally, we have to pay the invoice before we can confirm payment."

"pay invoice" -> F("confirm payment")

"If a request is sent, then a proposal is expected to be prepared afterwards."

"send request" -> F("prepare proposal")

Please note that these LTL constraints are simplified based on the context given and the limited set of constraints you provided. The actual implementation may vary based on the specific requirements of your system.

Prompt:
now these 10:

A confirmation is supposed to be mandatorily given after the proposal, and vice-versa any proposal is expected to precede a confirmation

Every process instance has to start by registering client data

Every provided room service must be billed.

Every provided laundry service must be billed

When the client leaves, the bill must be charged

The Police Report related to the car accident is searched within the Police Report database and put in a file together with the Claim Documentation.

A customer brings in a defective computer and the manager analyzes the defect and gives back an invoice.

After each of these activities, the proper system functionality is tested.

If an error is detected, another arbitrary repair activity is executed, otherwise the repair is finished.

After a claim is registered, it is examined by a claims officer.

Response:
Here are your sentences translated into LTL constraints:

"A confirmation is supposed to be mandatorily given after the proposal, and vice-versa any proposal is expected to precede a confirmation."

"prepare proposal" -> F("give confirmation") AND NOT("give confirmation") U "prepare proposal"

"Every process instance has to start by registering client data."

START -> "register client data"

"Every provided room service must be billed."

"provide room service" -> F("bill")

"Every provided laundry service must be billed."

"provide laundry service" -> F("bill")

"When the client leaves, the bill must be charged."

"leave" -> F("charge bill")

"The Police Report related to the car accident is searched within the Police Report database and put in a file together with the Claim Documentation."

"search report" -> F("put in a file together with the claim documentation")

"A customer brings in a defective computer and the manager analyzes the defect and gives back an invoice."

"bring in a defective computer" -> F("analyze the defect") AND F("give back invoice")

"After each of these activities, the proper system functionality is tested."

"each of these activities" -> F("test proper system functionality")

"If an error is detected, another arbitrary repair activity is executed, otherwise the repair is finished."

"error is detected" -> F("execute repair activity")

"After a claim is registered, it is examined by a claims officer."

"register claim" -> F("examine claim")

Please note that these LTL constraints are simplified based on the context given and the limited set of constraints you provided. The actual implementation may vary based on the specific requirements of your system.

Prompt:
now these 10:

This recommendation is then checked by a senior claims officer who may mark the claim as OK or Not OK.

If the claim is marked as Not OK, it is sent back to the claims officer and the recommendation is repeated.

If the claim is OK, the claim handling process proceeds.

The process of buying new hardware is started as soon as a request for new hardware is received by the IT department.

The next step is for the IT department to analyse the request and either approve or deny it.

If it is denied, the employee is informed via email about the denial and the process is ended.

However, if the request is approved it is edited and completed.

Once this offer is received, a form is filled in and sent to the management.

The management then analyses the request and decides if it is approved or not.

When this is accepted and the financial resources are found, the hardware is received by the IT department.

Response:
Here are your sentences translated into LTL constraints:

"This recommendation is then checked by a senior claims officer who may mark the claim as OK or Not OK."

"Check recommendation" -> F("Mark as Ok") OR F("Mark as not ok")

"If the claim is marked as Not OK, it is sent back to the claims officer and the recommendation is repeated."

"Mark as not ok" -> F("Check recommendation")

"If the claim is OK, the claim handling process proceeds."

"Mark as Ok" -> F("Proceed with claim")

"The process of buying new hardware is started as soon as a request for new hardware is received by the IT department."

START -> "Receive request for new hardware"

"The next step is for the IT department to analyse the request and either approve or deny it."

"Receive request for new hardware" -> F("Analyze request") AND (F("Approve request") OR F("Deny request"))

"If it is denied, the employee is informed via email about the denial and the process is ended."

"Deny request" -> F("inform employee via email")

"However, if the request is approved it is edited and completed."

"Approve request" -> F("edit request")

"Once this offer is received, a form is filled in and sent to the management."

"Receive offer" -> F("fill in form")

"The management then analyses the request and decides if it is approved or not."

"Receive request" -> F("Analyze request") AND (F("decide if approved") OR F("decide invoice legit"))

"When this is accepted and the financial resources are found, the hardware is received by the IT department."

"this is accepted" -> F("receive hardware by IT department")

Please note that these LTL constraints are simplified based on the context given and the limited set of constraints you provided. The actual implementation may vary based on the specific requirements of your system.

Prompt:
now these 10:After which the financial department will take care of the invoice and the processes is ended.

In the scenario where the management does not approve the request, it can send the request back to the IT department for rework, then the IT department has to go back to editing and completing a request.

If the management does not approve the request, it can also decide to deny the request after which the process is ended.

The loan approval process starts by receiving a customer request for a loan amount.

If the customer needs further review or the loan amount is for \$10,000 or more, the request is sent to the approver Web service.

The process of an Office Supply Request starts when any employee of the organization submits an office supply request.

Once the requirement is registered, the request is received by the immediate supervisor of the employee requesting the office supplies.

If the request is rejected, the process will end.

If the request is asked to make a change, then it is returned to the petitioner who can review the comments for the change request.

If the request is approved, it will go to the purchase department that will be in charge of making quotations (using a subprocess) and select a vendor.

Response:
Here are your sentences translated into LTL constraints:

"After which the financial department will take care of the invoice and the processes is ended."

"take care of invoice" -> F("END")

"In the scenario where the management does not approve the request, it can send the request back to the IT department for rework, then the IT department has to go back to editing and completing a request."

NOT("decide if approved") -> F("edit request")

"If the management does not approve the request, it can also decide to deny the request after which the process is ended."

NOT("decide if approved") -> F("Deny request") AND F("END")

"The loan approval process starts by receiving a customer request for a loan amount."

START -> "receive customer request"

"If the customer needs further review or the loan amount is for $10,000 or more, the request is sent to the approver Web service."

"need further review" -> F("send request")

"The process of an Office Supply Request starts when any employee of the organization submits an office supply request."

START -> "submit office supply request"

"Once the requirement is registered, the request is received by the immediate supervisor of the employee requesting the office supplies."

"register requirement" -> F("receive request")

"If the request is rejected, the process will end."

"reject request" -> F("END")

"If the request is asked to make a change, then it is returned to the petitioner who can review the comments for the change request."

"ask to make change" -> F("return to petitioner/employee")

"If the request is approved, it will go to the purchase department that will be in charge of making quotations (using a subprocess) and select a vendor."

"Approve request" -> F("go to purchase department") AND F("select vendor")

Please note that these LTL constraints are simplified based on the context given and the limited set of constraints you provided. The actual implementation may vary based on the specific requirements of your system.

Prompt:
now these 10:

After a vendor is selected and confirmed, the system will generate and send a purchase order and wait for the product to be delivered and the invoice received.

In any of the cases, approval, rejection or change required the system will send the user a notification.

The process starts when any employee of the organization submits a vacation request.

Once the requirement is registered, the request is received by the immediate supervisor of the employee requesting the vacation.

If the request is rejected, the application is returned to the applicant/employee who can review the rejection reasons.

If the request is approved, a notification is generated to the Human Resources Representative, who must complete the respective management procedures.

The process starts by reviewing the order.

If something is missing, the staff has to inform customer about a potential delay and order the missing products.

After waiting an average of x days for the delivery, the staff should check again if all products are on store.

The courier company informs the client about a potential delay until there are enough human resources to process the delivery.

Response:
Here are your sentences translated into LTL constraints:

"After a vendor is selected and confirmed, the system will generate and send a purchase order and wait for the product to be delivered and the invoice received."

"select vendor" -> F("generate purchase order") AND F("receive invoice")

"In any of the cases, approval, rejection or change required the system will send the user a notification."

"approval, rejection, change" -> F("send notification")

"The process starts when any employee of the organization submits a vacation request."

START -> "submit vacation request"

"Once the requirement is registered, the request is received by the immediate supervisor of the employee requesting the vacation."

"register requirement" -> F("receive request")

"If the request is rejected, the application is returned to the applicant/employee who can review the rejection reasons."

"reject request" -> F("return application")

"If the request is approved, a notification is generated to the Human Resources Representative, who must complete the respective management procedures."

"Approve request" -> F("generate notification")

"The process starts by reviewing the order."

START -> "review order"

"If something is missing, the staff has to inform customer about a potential delay and order the missing products."

"need further review" -> F("inform about delay") AND F("order missing products")

"After waiting an average of x days for the delivery, the staff should check again if all products are on store."

"wait for x days" -> F("check again")

"The courier company informs the client about a potential delay until there are enough human resources to process the delivery."

"inform about delay" -> F("process delivery")

Please note that these LTL constraints are simplified based on the context given and the limited set of constraints you provided. The actual implementation may vary based on the specific requirements of your system.

Prompt:
lastly these 14:

Once the driver delivers the package, she should update the system so all stakeholders are notified.

Whenever the sales department receives an order, a new process instance is created.

If the storehouse has successfully reserved or back-ordered every item of the part list and the preparation activity has finished, the engineering department assembles the bicycle.

Afterwards, the sales department ships the bicycle to the customer and finishes the process instance.

If the requested amount is lower or equal to 1M$, the company assess the risk of the credit.

After the approval request, the credit could be accepted or rejected

If the requested amount is greater than 1M$, an approval must be requested.

If the requested amount is lower or equal to 1M$, the company assess the risk of the credit.

As soon as an employee files an expense report, the process is started.

Once the expense report is received, a check is performed to make sure the employee has an account.

If this check does not take place within 28 days, the process is stopped and the employee is sent a submit email.

First, the supervisor approves within 7 days, this will result in the deposit of the reimbursement directly to the employee's bank account.

Second, the supervisor rejects within 7 days, as a result the employee receives a notification through email that the request has been rejected.

Third, if it takes the supervisor more than 7 days to respond, an email is sent to the employee informing him/her that the approval is in progress and the part of the processes after a supervisor is asked for approval is repeated.

Response:
Here are your sentences translated into LTL constraints:

"Once the driver delivers the package, she should update the system so all stakeholders are notified."

"deliver package" -> F("update system")

"Whenever the sales department receives an order, a new process instance is created."

"Receive order" -> F("create case")

"If the storehouse has successfully reserved or back-ordered every item of the part list and the preparation activity has finished, the engineering department assembles the bicycle."

"reserve part" -> F("assemble bicycle")

"Afterwards, the sales department ships the bicycle to the customer and finishes the process instance."

"assemble bicycle" -> F("ship bicycle to the customer") AND F("close case")

"If the requested amount is lower or equal to 1M$, the company assess the risk of the credit."

"Receive amount request" AND "Request approval" -> F("Assess credit risk")

"After the approval request, the credit could be accepted or rejected"

"Request approval" -> F("Accept credit") OR F("Perform check")

"If the requested amount is greater than 1M$, an approval must be requested."

"Receive amount request" -> F("Request approval")

"As soon as an employee files an expense report, the process is started."

"File expense" -> START

"Once the expense report is received, a check is performed to make sure the employee has an account."

"Receive expense report" -> F("Perform check")

"If this check does not take place within 28 days, the process is stopped and the employee is sent a submit email."

NOT("Perform check") U "stop process" AND "stop process" -> F("send email")

"First, the supervisor approves within 7 days, this will result in the deposit of the reimbursement directly to the employee's bank account."

"supervisor approves" -> F("deposit of reimbursement")

"Second, the supervisor rejects within 7 days, as a result the employee receives a notification through email that the request has been rejected."

"supervisor rejects" -> F("receives a notification")

"Third, if it takes the supervisor more than 7 days to respond, an email is sent to the employee informing him/her that the approval is in progress and the part of the processes after a supervisor is asked for approval is repeated."

NOT("respond") U "send email"

Please note that these LTL constraints are simplified based on the context given and the limited set of constraints you provided. The actual implementation may vary based on the specific requirements of your system.

Prompt:
you forgot this one:

If the requested amount is lower or equal to 1M$, the company assess the risk of the credit.

Response:
My apologies for that oversight. Here is the translation:

"If the requested amount is lower or equal to 1M$, the company assess the risk of the credit."

"Receive amount request" AND "Request approval" -> F("Assess credit risk")

Please note, in this example, I am assuming that "Receive amount request" and "Request approval" are the actions that indicate an amount request of 1M$ or less. You may need to adjust the translation based on your actual process.